LF-MC: An Efficient Verifier for Lingua Franca

Mario Reja[1], Mikheil Rukhaia[2], Kyungmin Bae[3], Mircea Marin[1], and Peter Csaba Ölveczky[4]

[1] West University of Timişoara, Timişoara, Romania
[2] Institute of Applied Mathematics, Tbilisi State University, Tbilisi, Georgia
[3] Pohang University of Science and Technology, Pohang, South Korea
[4] University of Oslo, Oslo, Norway

Abstract. This tool paper presents the LF-MC verification tool for the Lingua Franca (LF) coordination language for cyber-physical systems. LF-MC is the only such tool which is based on LF's logical-time semantics, and therefore gives correct analysis results. LF-MC is also the only verifier for LF that supports a model of nondeterministic environments. LF-MC is fully integrated into the LF toolchain, and provides system simulation as well as fully automatic unbounded and time-bounded reachability analysis and LTL model checking for both state-based and action-based properties. Extensive benchmarking shows that LF-MC significantly outperforms the LF Verifier and PolyVer analysis tools for LF.

1 Introduction

Lingua Franca (LF) [7] is a recent reactor-oriented "coordination language for efficient, deterministic, multi-threaded, time-sensitive, embedded, and distributed programs" developed by the group of Edward A. Lee at UC Berkeley [5].

An LF program consists of *reactors*—with state variables, input and output ports, actions, timers, and a list of *reactions*—and immediate and delayed connections. A reaction executes *reaction code* when triggered by actions, inputs, or timers. The reaction code is given in a target language such as C or Rust, hence "Lingua Franca," and may update state variables, produce outputs, and/or schedule future actions. Reaction executions are considered to be *instantaneous*, so that a reaction's output traveling along an *immediate* connection may trigger *another* reaction execution *at the same logical time*, and so on. LF has a *logical-time semantics* [7,4,9]: all reactions executed at the same logical time happen in *one* step, where the order of reaction executions satisfies certain constraints to ensure *deterministic* behavior. *Physical actions* model external environments, and are nondeterministic in *when* they take place and *what* "values" they have.

Providing an executable formalization of LF's logical-time semantics is known to be challenging (e.g., [4]). In our paper [12] we provide the first such formalization, for a significant subset of LF, in rewriting logic [10], and show how Maude [3] and Real-Time Maude [11] can be used to formally analyze LF programs.

Other approaches to verifying LF programs [16,4,2] use an *event-based* semantics, in which *each* reaction execution corresponds to a transition. Analyses based

on this semantics explore additional "intermediate" states not reachable with a logical-time semantics. In contrast, in our logical-time semantics *all* reactions executed at the same logical time *together* define a single transition.

Unlike other verification approaches for LF, we also support a model of non-deterministic physical actions (while others model check deterministic programs).

The LF-MC tool presented in this paper uses our formalization of the LF semantics to support automatic Maude-based formal analysis of LF programs. LF-MC is fully integrated into the LF toolchain: The user provides an LF program annotated with desired properties; LF-MC then generates the Maude model corresponding to the LF program, invokes Maude to execute analysis commands checking the user's properties, and outputs the analysis in readable form.

Our paper [12][5] devotes 1+ page to LF-MC, but does not provide benchmarking, description of implementation, comparison of supported features, etc.

LF-MC (version 1.2) is distributed as an artifact [15] and a Docker image, for `amd64` and `arm64`, at `ghcr.io/symbolicsafety/lf-mc:v1.2`. The image bundles the two main components of LF-MC: a modified LF compiler `lfc` and the Maude run-time. The annotated `.lf` programs used in our evaluation are included under `/opt/lf-maude/examples/src/`. The source code is available at `github.com/symbolicsafety/lf-maude`, which contains Maude-specific components, and `github.com/symbolicsafety/lf-mc`, which hosts the Java-based LF compiler extensions.

2 Lingua Franca

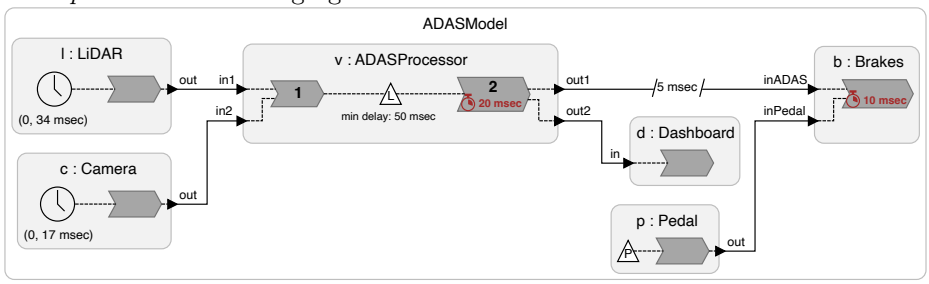
Lingua Franca (LF) [7,9,5] is a coordination language for cyber-physical systems. LF supports *reactor-oriented* programming [8,6]: the system components are modeled by *reactors*, which are *deterministic* actors with local state variables, ports, actions, timers, and an ordered list of *reactions*. A reaction *invocation* (or execution) is triggered by, and may produce, *discrete events*. A triggering event can be (the presence of an event at) an input port, a timer, or an action. A *logical action* is used to schedule future events. A reaction has the form `reaction(`*triggers*`) ->` *effects* `{=` *body* `=}`, where *triggers* is a set of triggers, *effects* are the ports and actions that the reaction *may* write to (resp., schedule), and *body* specifies the reaction's behavior in a *target language* supported by LF.

LF has a *logical-time* semantics: all reaction invocations at the same logical time happen in a *single step*. The reactions executed in this step may be executed in different orders, as long as a reaction r_1 whose output triggers a reaction r_2 is executed before r_2, and if reactions in the same reactor are executed at the same logical time, then they are executed in order of declaration.

Physical actions represent external events from the physical environment, the timing and values of which are not controlled by the program. LF distinguishes between events from the physical environment and events produced by the program by considering two time lines: *logical time* and *physical time*.

[5] An extended version of our TACAS'26 paper [12] has been submitted to a journal.

Example 1. The following figure



shows the architecture of the LF program below (ignore the `@maude...` annotations for now). It models a driver assistance system (ADAS). A `Camera c` and a `LiDAR l` are connected (with immediate connections) to an `ADASProcessor v`, which has a delayed connection to the `Brakes` reactor `b`. The `LiDAR` and the `Camera` are triggered periodically by *timers* with respective periods 34 and 17. Reaction **1** in the `ADASProcessor` schedules a logical ('`L`') action (that will trigger reaction **2**) with delay 50 msec. The driver can press the brake pedal at any time; this is modeled by the physical ('`P`') action `a` in the `Pedal` component.

```
target C;
reactor Camera { output out:int;   state  frame:int;   timer  t(0, 17 msec);
                 reaction(t) -> out {= self->frame = 1;  lf_set(out, self->frame); =} }
reactor LiDAR { output out:int;   state  frame:int;   timer  t(0, 34 msec);
                reaction(t) -> out {= self->frame = 2;  lf_set(out, self->frame); =} }
reactor Pedal { output out:int;   physical action a;
                reaction(a) -> out {= lf_set(out, 1); =} }
reactor Brakes { input inADAS:int;   input inPedal:int;   state brakesApplied:int = 0;
                 reaction(inADAS, inPedal) {= self->brakesApplied = 1; =} }
reactor ADASProcessor {input in1:int; input in2:int; output out1:int; output out2:int;
                       logical action a(50 msec);    state requestStop:int;
         reaction(in1,in2) -> a {= lf_schedule(a, 0);  self->requestStop = 1; =}
         reaction(a) -> out1, out2 {= if (self->requestStop == 1) lf_set(out1, 1); =} }
reactor Dashboard { input in:int;  state received:int;
                    reaction(in) {= self->received = 1; =} }
main reactor ADASModel { c = new Camera();  l = new LiDAR();  v = new ADASProcessor();
                         b = new Brakes();  d = new Dashboard();  p = new Pedal();
                         l.out -> v.in1;   c.out -> v.in2;  v.out2 -> d.in;
                         v.out1 -> b.inADAS after 5 msec;   p.out -> b.inPedal; }

@maudePhysAct(name="a", inReactor="p", vals="69, 71", period=5, timeNonDet=true)
@maude(analysis="simulation", rewrites=51)
@maude(analysis="reachability", timeBound=5, goal="brakesApplied in b == 1")
@maude(analysis="ltl", timeBound=55, goal="<>(brakesApplied in b == 1)")
@maude(analysis="ltl", timeBound=54, goal="<>(brakesApplied in b == 1)")
@maude(analysis="ltl",  goal="[]((l.1 invoked /\ requestStop in v == 1) ->
                                 (<> brakesApplied in b == 1))")
@maude(analysis="reachability", goal="(p.1 invoked) /\ (brakesApplied in b != 1)")   □
```

3 Related Work on Verifying LINGUA FRANCA Programs

Sirjani et al. [16] "hand translate" three LF programs to *an extension* of Timed Rebeca [1]. This yields an *event-based* semantics, and they perform exhaustive state space analysis. Physical actions are translated into deterministic behaviors.

The LF Verifier [4] also provides an *event-based* semantics. The tool supports safety metric temporal logic properties, where all temporal operators have upper bounds (e.g., $\Box_{[2,5]}(p \rightarrow \Diamond_{[1,6]}\, q)$), so that it is sufficient to perform *bounded* model checking (e.g., up to time 11 for the above formula), and uses Uclid5 [14] to compile such *bounded* model checking to an SMT problem to be solved by Z3. LF Verifier supports both state-based and action-based ("dataflow") properties, but does *not* support physical actions. The PolyVer tool [2] extends this approach to *polyglot* LF programs, where the reaction code of different reactors can be given in different languages (currently C and Rust).

4 The LF-mc Verification Tool

The LF-mc tool extends the LF compiler (`lfc`) to:

1. generate a Maude model from the LF code, annotated with intuitive queries,
2. generate Maude analysis commands for the user queries,
3. invoke Maude to execute the analysis commands on the Maude model, and
4. output the results of the analysis commands in an understandable form.

LF-mc supports a fragment of LF comparable to that of LF Verifier (see Table 1), extended with the following model for physical actions: Each physical action has a user-defined period and a finite range of possible values. At the end of each period, the physical action *may* or *must* take place (depending on the `timeNonDet` flag); if it takes place, its value is selected nondeterministically from the finite set. The user should add to her LF program annotations of the form

```
@maudePhysAct(name="action", inReactor="reactor", vals="values", period=p, timeNonDet=b)
```

to specify these parameters of physical actions, where *values* can be written either as an integer interval (e.g., `2 .. 5`) or as a comma-separated list of (Boolean or integer) values. LF-mc captures all 22 LF Verifier benchmarks and all four LF programs in [4,16] (which all have physical actions!).

LF-mc supports unbounded and time-bounded simulation, reachability analysis, and LTL model checking, for both state and action ("dataflow") properties.

Table 1 compares the LF features and analysis capabilities supported by our work, LF Verifier [4], the work in [16], and PolyVer [2]. "ub" and "tb" denote that, resp., *unbounded* and *time-bounded* analyses are supported.

4.1 Property Specification Language

Desired properties are specified in LF source files using annotations of the form

```
@maude(analysis="simulation", timeBound=t,  rewrites=n)
@maude(analysis="reachability", timeBound=t,  rewrites=n, goal="stateFormula")
@maude(analysis="ltl", timeBound=t,  goal="LTLformula")
```

Table 1. Comparison of supported features

Feature	LF-MC	LF Verifier	[16]	PolyVer
LF Features				
Logical-time semantics	✓	×	×	×
Two timelines	×	×	×	×
Nondeterministic physical actions	✓	×	×	×
Logical actions	✓	✓	×	✓
Minimum spacing of actions	×	×	×	×
Multiple reaction triggers	✓	✓	×	✓
Verification Capabilities				
Reachability analysis	ub+tb	×	ub?	×
LTL model checking	ub+tb	×	×	×
Timed temporal logic model checking	ub+tb	tb	×	tb
State properties	✓	✓	✓	✓
Dataflow properties	✓	✓	×	✓
Tool Integration				
Automated model and analysis command generation	✓[9]	✓	×	✓
Native languages supported	C	C	—	C, Rust

`timeBound` and `rewrites` (a bound on the number of rewrite steps) are optional.

LF-MC supports the following analyses: (i) `simulation` performs bounded simulation and prints the resulting state; (ii) `reachability` outputs the path to a reachable state (if any) satisfying a given state formula; and (iii) `ltl` performs LTL model checking, and outputs a readable counterexample path if the property does not hold. The reachability annotation may also include the attribute `type="!"` to restrict the search to deadlocked (final) states.

A *stateFormula* is a Boolean combination (using `!` or `~` for negation, `/\` for conjunction, `\/` for disjunction, `->` for implication, etc.) of *state propositions*. An *LTLformula* extends state formulas to LTL formulas by allowing the temporal operators `[]`, `<>`, `U`, `W` (weak until), and `O` (next). State propositions are:

- r `.` n `invoked`, which holds if the nth reaction in reactor r was invoked in the big step leading to the current state;
- `event(`r`,` *trigger*`,` *val*`) inQueue`, where *val* is optional, which holds if the event queue contains the/an event `event(`r`,` *trigger*`,` *val*`)`;
- a *relational expression* over reactor variables and input ports;
- `remainingTime` t, which holds when the remaining time until the specified time bound is t (only for time-bounded analysis queries).

Relational expressions are built from Boolean or integer constants, *var* `in` r (the value of variable *var* in reactor r), and *port* `in` r (the value of input port *port* in reactor r). These expressions may be combined using arithmetic operators (e.g., `+`, `-`, `*`) and relational operators (e.g., `==`, `!=`, and `>=`, `>`, `<=`, and `<` for integers).

To analyze a `.lf` file containing an annotated LF program, LF-MC can be invoked from the command line as follows: `lfc` *path/fileName*`.lf`

LF-MC prints LTL counterexamples and search paths in an intuitive form. In each state, only the event queue, the values of reactor variables and input ports, and the reactions executed in the last step are retained, as shown in Appendix A.

4.2 Architecture and Implementation

LF-MC extends the LF compiler. This allows us to inherit parsing, name resolution, and attribute validation from the LF compiler. This enables a semantics-based model synthesis rather than ad-hoc source-to-source translation. The tool is composed of two parts: a front-end, written in Java, and a backend, written in Maude, that performs the formal verification. The front-end generates a `.maude` file containing both the translated LF program and the analysis commands, and invokes the Maude backend to execute these commands and return the results.

To translate annotated LF programs into Maude code and commands, we reuse existing LF infrastructure. This includes using ANTLR [13] as the parser generator for LF and reaction code, as well as for parsing analysis annotations.

Our implementation extends the Java classes in the LF implementation used to represent reactors, reactions, actions, timers etc. with naming, typing and other necessary information required by the Maude backend. We follow the way the UCLID5 transformation was implemented and provide the `MaudeGenerator` and `MaudeRunner` classes. We use the `CodeBuilder` object (already implemented inside the LF framework) to generate Maude files.

4.3 Benchmarking

We compare the performance of LF-MC with LF VERIFIER and POLYVER on the LF VERIFIER benchmark suite, which consists of 22 LF programs drawn from real-world applications and other benchmarks suites. LF-MC could successfully verify the desired—or stronger—properties of all programs in this suite.

We were unable to reproduce the results reported for LF VERIFIER from the publicly available artifact, but use the numbers reported in [4], and likewise use the runtime results from [2] for POLYVER. All LF-MC experiments were executed using the Docker image `lf-mc:v1.2`, on a laptop running macOS 15.7.4, equipped with a 3.2 GHz Apple M1 Max processor (10 cores: 8 performance and 2 efficiency) and 32GB of RAM. POLYVER ran its benchmarks using a single core of a 3.7 GHz Intel Core i9 machine with 62 GB RAM. LF VERIFIER used a macOS laptop with a 2.3 GHz 8-core Intel Core i9 and 16 GB RAM.

Table 2 summarizes the experimental results. For the LF VERIFIER, "Gen." indicates the time taken to generate SMT encodings, and "Solving" reports the time spent on SMT solving. For LF-MC, "Gen." shows the time needed to parse the annotated `.lf` file, produce a `.maude` file, and invoke Maude; and "Solving" shows the time needed by Maude to analyze the properties. For POLYVER, "SOT" presents the synthesis oracle time, "VOT" the verification oracle time and "UT" is the UCLID5 total time. All execution times are in seconds.

LF-MC drastically outperforms both LF VERIFIER and POLYVER, possibly because LF-MC executes many reactions in one step, resulting in fewer states. Since LF is *deterministic*, it is sufficient to execute all reactions in a single "big step" in *some* allowed order, whereas event-based model checkers, such as LF VERIFIER, explore *all* such sequences of reaction executions, which can be exponential in the number of reactions executed.

Table 2. LF Verifier vs. LF-MC vs. PolyVer

Program	LF Verifier			LF-MC			PolyVer			
	Gen.	Solving	Total	Gen.	Solving	Total	SOT	VOT	UT	Total
ADAS	21.07	1.72	22.79	1.775	0.002	1.777	220.9	3.2	21.5	245.6
AircraftDoor	6.57	0.02	6.59	1.699	<0.001	1.699	82.1	1.2	2.0	85.3
Alarm	5.36	0.01	5.37	1.663	<0.001	1.663	62.3	0.9	2.1	65.3
CoopSchedule	21.1	0.31	21.41	1.686	0.009	1.695	54.5	0.8	13.4	68.7
Elevator	325.14	1038.02	1363.16	1.747	0.002	1.749	413.1	33.4	664.8	1111.3
Election	41.65	3.27	44.92	1.716	<0.001	1.716	124.2	1.4	83.9	209.5
Election2	12.21	0.17	12.38	1.754	<0.001	1.754	142.8	1.5	11.7	156.0
Factorial	21.46	13.15	34.61	1.694	0.01	1.704	138.2	1.9	63.0	203.1
Fibonacci	94.96	153.49	248.45	1.694	0.01	1.704	167.3	2.4	390.9	560.6
PingPong	13.42	0.14	13.56	1.688	<0.001	1.688	131.0	1.8	8.2	141.0
Pipe (I)	142.41	12.15	154.56	1.717	0.001	1.718	147.4	2.4	172.2	322.0
ProcessMsg	13.23	0.74	13.97	1.681	0.008	1.689	116.6	1.8	5.1	123.5
ProcessSync	4.99	0.01	5.00	1.681	<0.001	1.681	35.2	0.4	1.7	37.3
Railroad	158.14	4.33	162.47	1.751	0.01	1.761	678.9	9.0	264.8	952.7
Ring	15.83	0.07	15.90	1.754	<0.001	1.754	105.6	1.8	6.8	114.2
RoadsideUnit	74.07	0.62	74.69	1.713	<0.001	1.713	259.0	3.8	25.3	288.1
SafeSend	6.48	0.03	6.51	1.697	<0.001	1.697	124.3	1.8	2.3	128.4
Subway	29.96	0.31	30.27	1.704	<0.001	1.704	266.5	4.5	67.2	338.2
Thermostat	13.41	0.14	13.55	1.587	<0.001	1.587	192.6	2.6	3.5	198.7
TrafficLight	-	-	-	1.708	<0.001	1.708	462.1	4.4	216.1	682.6
TrainDoor	5.65	0.01	5.66	1.704	<0.001	1.704	74.5	1.2	2.1	77.8
UnsafeSend	6.5	0.02	6.52	1.703	<0.001	1.703	122.1	1.8	2.9	126.8

5 Concluding Remarks

LF-MC is the only verification tool for Lingua Franca (LF) that is based on LF's *logical-time* semantics. LF-MC is fully automatic and integrated into the LF toolchain: the user enters an LF program annotated with desired properties, and LF-MC calls Maude to perform the corresponding analysis and displays the analysis results in a readable form. LF-MC is also the only verifier for LF which supports a model of nondeterministic physical actions.

LF-MC supports unbounded and time-bounded simulation, reachability analysis, and LTL model checking for both state- and action-based properties. Using the correct semantics (and more appropriate analysis tools?) gives the correct analysis results *and* much more efficient analysis, as our benchmarking shows.

A key take-away message from this work is the suitability of using rewriting logic and Maude to formalize the semantics and provide analysis for sophisticated (real-time) languages. Formalizing the LF semantics was manageable in rewriting logic, which uses rewrite rules (for transitions) and equations (to define functions to, e.g., perform a big step). This task seems impossible using, e.g., timed automata or time(d) Petri nets (because of C programs, complex big steps, and unbounded event queues) or actor-based languages such as Timed Rebeca. It also seems very hard to encode model checking for the correct LF semantics as SMT formulas. Finally, Maude comes with purpose-made reachability and LTL model checking implementations, which should be more efficient than encoding model checking as SMT problems.

References

1. Aceto, L., Cimini, M., Ingólfsdóttir, A., Reynisson, A.H., Sigurdarson, S.H., Sirjani, M.: Modelling and simulation of asynchronous real-time systems using Timed Rebeca. In: 10th International Workshop on the Foundations of Coordination Languages and Software Architectures (FOCLASA 2011). EPTCS, vol. 58, pp. 1–19 (2011)
2. Chen, P.W., Lin, S., Godbole, A., Singh, R., Polgreen, E., Lee, E., Seshia, S.: PolyVer: A compositional approach for polyglot system modeling and verification. In: Proceedings of the 25th Conference on Formal Methods in Computer-Aided Design – FMCAD 2025. pp. 1–13. TU Wien Academic Press, Vienna, Austria (2025)
3. Clavel, M., Durán, F., Eker, S., Meseguer, J., Lincoln, P., Martı-Oliet, N., Talcott, C.: All About Maude – A High-Performance Logical Framework, LNCS, vol. 4350. Springer, Berlin, Heidelberg (2007)
4. Lin, S., Manerkar, Y.A., Lohstroh, M., Polgreen, E., Yu, S.J., Jerad, C., Lee, E.A., Seshia, S.A.: Towards Building Verifiable CPS using Lingua Franca. ACM Transactions on Embedded Computing Systems **22**(5s), 1–24 (2023)
5. `https://www.lf-lang.org/`, accessed March 20, 2026
6. Lohstroh, M.: Reactors: A Deterministic Model of Concurrent Computation for Reactive Systems. Ph.D. thesis, University of California, Berkeley (2020)
7. Lohstroh, M., Menard, C., Bateni, S., Lee, E.A.: Toward a Lingua Franca for Deterministic Concurrent Systems. ACM Transactions on Embedded Computing Systems (TECS) **20**(4), 1–27 (2021)
8. Lohstroh, M., Schoeberl, M., Goens, A., Wasicek, A., Gill, C., Sirjani, M., Lee, E.A.: Actors revisited for time-critical systems. In: Proceedings of the 56th Design Automation Conference 2019 (DAC'19). pp. 152:1–152:4. ACM, New York, NY, USA (2019)
9. Menard, C., et al.: High-performance deterministic concurrency using Lingua Franca. ACM Transactions on Architecture and Code Optimization **20**(4), 1–29 (2023)
10. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical Computer Science **96**(1), 73–155 (1992)
11. Ölveczky, P.C.: Real-Time Maude and its applications. In: Proc. WRLA'14. LNCS, vol. 8663, pp. 42–79. Springer, Cham (2014)
12. Ölveczky, P.C., Reja, M., Rukhaia, M., Bae, K., Marin, M.: Efficient verification of Lingua Franca programs. In: Proc. TACAS'26. LNCS, vol. 16506, pp. 23–41. Springer (2026)
13. Parr, T.: The definitive ANTLR 4 reference. The Pragmatic Bookshelf, Dallas, Texas (2013), `https://www.antlr.org/`
14. Polgreen, E., Cheang, K., Gaddamadugu, P., Godbole, A., Laeufer, K., Lin, S., Manerkar, Y.A., Mora, F., Seshia, S.A.: UCLID5: multi-modal formal modeling, verification, and synthesis. In: Proc. CAV'22. LNCS, vol. 13371, pp. 538–551. Springer, Cham (2022)
15. Reja, M., Rukhaia, M., Bae, K., Marin, M., Ölveczky, P.C.: Artifact for "LF-MC: An efficient verifier for Lingua Franca," submitted to QEST+FORMATS 2026. `https://doi.org/10.5281/zenodo.19568831` (2026)
16. Sirjani, M., Lee, E.A., Khamespanah, E.: Verification of cyberphysical systems. Mathematics **8**(7), 1068 (2020)

A An Example

The LF program in Section 2 ends with seven LF-MC annotations. The first one specifies that the physical action `a` in the `Pedal` reactor `p` has period 5, possible values 69 and 71, and may or may not take place when its period ends (`timeNonDet=true`). The other annotations specify the following analysis commands: (1) simulate the system for (up to) 51 rewrite steps; (2) check whether it is possible to reach a state within time 5 in which the state variable `brakesApplied` in the reactor `b` has value 1; (3) check whether one can *always* reach such a state within time 55; (4) the same, but with time bound 54; (5) check whether any state, reached in a big step where the first (and only) reaction in the `LiDAR` reactor `l` was executed, where the value of the variable `requestStop` is 1, must imply that `brakesApplied` sooner or later will be 1; and (6) check whether it is possible to reach a *bad state*, in which the first/only (physical) action in `p` was invoked, but where `brakesApplied` is different from 1. The last two properties have no time bound.

The following shows (parts of) the result of running LF-MC on this annotated LF program:[6]

```
linux> lfc lf-maude/examples/src/ADAS-model.lf
[...]
==========================================
rewrite [51] in SIMULATION-ADASMODEL : initSystem timeBound INF .
rewrites: 3949 in 2ms cpu (2ms real) (1320735 rewrites/second)
result ClockedSystem: {...}  in time 75000000 timeBound INF
==========================================
reduce in ANALYSIS-ADASMODEL : search in 'ANALYSIS-ADASMODEL : initSystem timeBound 5000000
    =>* (@ b.sv.brakesApplied in b === @ [1]) .
rewrites: 772 in 11ms cpu (12ms real) (64912 rewrites/second)
result SearchOutput: trace
 { invoked: none
   queue: (event(c, c.t.t, [0]) event(l, l.t.t, [0])) at tag(0, 1)
   c :[ inports: empty   state: c.sv.frame |-> [0]]
   l :[ inports: empty   state: l.sv.frame |-> [0]]
   v :[ inports: (v.in.in1 |-> [0]) ; v.in.in2 |-> [0]    state: v.sv.requestStop |-> [0]]
   b :[ inports: (b.in.inADAS |-> [0]) ; b.in.inPedal |-> [0]   state: b.sv.brakesApplied |-> [0]]
   d :[ inports: d.in.in |-> [0]   state: d.sv.received |-> [0]]
   p :[ inports: empty    state: empty] }
=['step]=>  ...  =['step]=>
 { invoked: (b . 1) ; (p . 1)
   queue: ...
   c :[ inports: empty   state: c.sv.frame |-> [1]]
   l :[ inports: empty   state: l.sv.frame |-> [2]]
   v :[ inports: (v.in.in1 |-> [2]) ; v.in.in2 |-> [1]   state: v.sv.requestStop |-> [1]]
   b :[ inports: (b.in.inADAS |-> [0]) ; b.in.inPedal |-> [1]   state: b.sv.brakesApplied |-> [1]]
   d :[ inports: d.in.in |-> [0]   state: d.sv.received |-> [0]]
   p :[ inports: empty   state: empty] }
==========================================
reduce in MODELCHECKER-ADASMODEL : modelCheck(initSystem timeBound 55000000,
    <> @ b.sv.brakesApplied in b === @ [1]) .
rewrites: 6535 in 4ms cpu (4ms real) (1326095 rewrites/second)
result Bool: true
==========================================
reduce in MODELCHECKER-ADASMODEL : modelCheck(initSystem timeBound 54000000,
    <> @ b.sv.brakesApplied in b === @ [1]) .
rewrites: 3222 in 0ms cpu (1ms real) (3231695 rewrites/second)
```

[6] LF-MC assumes that time bounds in annotations are given in milliseconds and converts them to nanoseconds, which is the time unit used by the LF compiler.

```
result ModelCheckResult:
counterexample
 { invoked: none
   queue: (event(c, c.t.t, [0]) event(l, l.t.t, [0])) at tag(0, 1)  ... }
===[ 'step ]===>
 { invoked: (c . 1) ; (l . 1) ; (v . 1)  ... }
===[ 'tick ]===>  ...  ===[ 'tick ]===>
begin loop
 { invoked: none
   queue: ...
   c :[ inports: empty    state: c.sv.frame |-> [1] ]
   l :[ inports: empty   state: l.sv.frame |-> [2] ]
   v :[ inports: (v.in.in1 |-> [2]) ; v.in.in2 |-> [1]   state: v.sv.requestStop |-> [1] ]
   b :[ inports: (b.in.inADAS |-> [0]) ; b.in.inPedal |-> [0]   state: b.sv.brakesApplied |-> [0] ]
   d :[ inports: d.in.in |-> [0]    state: d.sv.received |-> [0] ]
   p :[ inports: empty    state: empty ] }
===[ deadlock ]===>
end loop
==========================================
reduce in MODELCHECKER-ADASMODEL : modelCheck(initSystem timeBound INF,
   []((l . 1) invoked /\ @ v.sv.requestStop in v === @ [1] -> <> @ b.sv.brakesApplied in b === @ [1])) .
rewrites: 81361 in 53ms cpu (54ms real) (1515158 rewrites/second)
result Bool: true
==========================================
reduce in ANALYSIS-ADASMODEL : search in 'ANALYSIS-ADASMODEL : initSystem timeBound INF
    =>* ((p . 1) invoked /\ @ b.sv.brakesApplied in b ==/= @ [1]) .
rewrites: 89546 in 63ms cpu (63ms real) (1409462 rewrites/second)
result SearchOutput:  No Solution
```

The output shows that a desired state is reachable in query (2) and that a bad state is unreachable in query (6). The LTL properties hold, except for the one where the brakes must be applied within time 54.